

بسته‌ی مطالعه‌ی LPIC-1 – آزمایشگاه ویژه (منتخب هفته ۲)

سرویس گزارش سلامت سرور

Permissions • Bash • systemd Service & Timer • Firewall • SSH • journald

بانک سؤال مصاحبه

تست با پاسخنامه

تئوری + آزمایشگاه

هفته ۲ (LPIC-1) • آزمایشگاه یکپارچه + آزمون‌های منتخب (۱۵ کوتاه + ۱۵ چندگزینه‌ای + ۱۰ مصاحبه) با پاسخنامه

آزمایشگاه ویژه (منتخب هفته ۲) – سرویس گزارش سلامت سرور

از صفر تا «سرویس آماده‌ی تولید»: مجوز • اسکریپت • systemd • فایروال • SSH • لاگ

این آزمایشگاه از دل روزهای هفته ۲ ساخته شده و همان‌ها را با هم تمرین می‌دهد: روز ۲ (کاربر، مجوز، ACL) • روز ۳ (اسکریپت قابل اتکا) • روز ۴ (زمان‌بندی، flock، nice) • روز ۵ (systemd، unit، Journal) • روز ۶ (فایروال، SSH کلیدمحور)
زمان تخمینی: ۹۰ تا ۱۲۰ دقیقه (هم‌راستا با اسلات ایوبینینگ ۱۷:۰۰-۲۰:۱۵) حداقل نیاز: یک VM لینوکس (Ubuntu 24.04 یا Rocky 9) با دسترسی root/sudo + یک ماشین کلاینت در همان شبکه.

۰) سناریو و صورت مسئله

شرکت «پارس‌داده» یک سرور داخلی دارد. تیم عملیات (ops) می‌خواهد بدون ورود به سرور، وضعیت سلامت آن را ببیند. از تو خواسته شده:

- یک کاربر غیرمجازمحور اختصاصی برای سرویس‌ها ساخته شود (بدون شل ورود، بدون دسترسی اضافه).
- یک اسکریپت گزارش سلامت هر ۱۰ دقیقه اجرا شود و فایل گزارش بسازد: uptime، load، حافظه، swap، دیسک‌های بالای ۸۰٪، سرویس‌های failed و پورت‌های گوش‌دهنده.
- گزارش‌ها در `/srv/health` باشد، فقط تیم ops آن را بخواند، و هر ۱۰ دقیقه خودکار به‌روز شود (با `systemd timer`، مقاوم به قطع برق/خاموشی سرور).
- یک داشبورد وب فقط‌خواندنی روی پورت ۸۰۸۰ همان گزارش را نمایش دهد.
- دسترسی‌ها قفل شود: SSH فقط با کلید، پورت ۸۰۸۰ فقط از شبکه‌ی مدیریتی `10.20.0.0/24`، بقیه drop.
- همه‌چیز قابل عیب‌یابی و قابل اتکا باشد: لاگ در journald، محدودیت منابع، ری‌استارت خودکار.

جدول تحویل (Deliverables):

#	خروجی	معیار
۱	کاربر healthsvc (سیستمی، nologin) + گروه healthops	id healthsvc مقادیر درست بدهد
۲	/opt/health/bin/health-report.sh	اجرای دستی موفق + مجوز اجرایی
۳	health-report.timer + health-report.service	list-timers زمان اجرای بعدی را نشان دهد
۴	health-http.service روی ۸۰۸۰	curl از شبکه‌ی مدیریت ۲۰۰ بدهد
۵	/srv/health/report.html با مالکیت و مجوز درست	ls -l و getfacl تأیید کنند
۶	سیاست فایروال nftables	فقط ۲۲ (محدود) و ۸۰۸۰ (محدود) باز باشد
۷	SSH کلیدمحرور	ورود با رمز رد شود

توپولوژی (ساده، دو ماشین):

```
[10.20.0.50 | کلاینت/ادمین] ---- SSH:22 ----> +-----+
(شبکه مدیریت) | SRV1 10.20.0.10/24 |
| Ubuntu/Rocky |
[192.168.50.20 | کلاینت نامجاز] -- 8080 -X-> | (وب) | سرویس: 8080
(شبکه دیگر/خارج) +-----+
|
| /srv/health/report.html
```

۱) گام ۱ – کاربر، گروه و مسیرها (روز ۲)

```
# گروه سرویس + کاربر سیستمی بدون شل ورود (بدون خانه‌ی تعاملی)
sudo groupadd -r healthops
sudo useradd -r -m -d /opt/health -g healthops -s /usr/sbin/nologin healthsvc
id healthsvc
# انتظار: uid (مثلا 990), gid healthops, shell nologin

# گروه تیم عملیات (اگر از قبل نیست)
sudo groupadd ops || true
sudo usermod -aG ops alice # alice = کاربر واقعی تیم (جای نام خودت بگذار)

# مسیرها
sudo mkdir -p /opt/health/bin /srv/health
sudo chown -R healthsvc:healthops /opt/health /srv/health
sudo chmod 750 /opt/health /opt/health/bin /srv/health

# خواندن و نوشتن بازنه‌ی ops پیش‌فرض تا فایل‌های جدید هم برای ACL فقط خواندن؛ تیم ops
sudo setfacl -R -m g:ops:r-x /srv/health
sudo setfacl -R -d -m g:ops:r-x /srv/health
getfacl /srv/health
```

چرا این‌طور؟

- `-r` (system user): UID پایین، بدون انقضای رمز، مناسب سرویس.
- `-s /usr/sbin/nologin`: حتی اگر رمزی تنظیم شود، کسی نمی‌تواند شل بگیرد (کاهش سطح حمله).
- `ACL` به جای `chmod 755`: می‌خواهیم فقط گروه `ops` بخواند، نه «همه».
- `ACL` پیش‌فرض (`-d`) یعنی فایل `report.html` که فردا ساخته می‌شود هم خودکار همان مجوز را بگیرد – نکته‌ای که در مصاحبه امتیاز دارد.

```

sudo tee /opt/health/bin/health-report.sh >/dev/null <<'SH'
#!/usr/bin/env bash
# health-report.sh – گزارش سلامت سرور برای داشبورد داخلی
# healthsvc (از طریق systemd timer) اجراکننده: کاربر
set -euo pipefail
IFS=$'\n\t'

OUT_DIR="/srv/health"
OUT_FILE="${OUT_DIR}/report.html"
LOCK_FILE="${OUT_DIR}/.report.lock"
TMP_FILE="${OUT_DIR}/.report.$$tmp"

cleanup() { rm -f "$TMP_FILE"; }
trap cleanup EXIT

# قفل: جلوگیری از اجرای همزمان (اگر اجرای قبلی طول بکشد)
exec 9>"$LOCK_FILE"
if ! flock -n 9; then
    echo "report already running; skipping this tick" >&2
    exit 0
fi

{
    printf '<!doctype html>\n<meta charset="utf-8">\n<title>%s health</title>\n' "$(hostname)"
    printf '<h1>گزارش سلامت %s</h1>\n' "$(hostname)"
    printf '<p>زمان تولید: %s</p>\n<pre>\n' "$(date +%F %T %Z)"
    printf 'uptime : %s\n' "$(uptime -p)"
    printf 'load : %s\n' "$(cut -d' ' -f1-3 /proc/loadavg)"
    printf 'cores : %s\n' "$(nproc)"
    printf 'mem : %s\n' "$(free -h | awk '/^Mem:/{print $3 / "$2" used}')"
    printf 'swap : %s\n' "$(free -h | awk '/^Swap:/{print $3 / "$2}')"
    echo '--- 80 % فایل‌سیستم‌های بالای ---'
    df -hP | awk 'NR>1 && int($5) >= 80 {print $6, $5, $4 " free"}'
    echo '--- سرویس‌های failed ---'
    systemctl --failed --no-legend | awk '{print $1}' || true
    echo '--- پورت‌های گوش‌دهنده ---'
    ss -tulnH | awk '{print $1, $5}' | sort -u
    printf '</pre>\n'
} > "$TMP_FILE"

chmod 640 "$TMP_FILE"
mv -f "$TMP_FILE" "$OUT_FILE" # جایگزینی اتمیک
trap - EXIT
echo "report written: ${OUT_FILE} ($(date +%T))"
SH

sudo chown healthsvc:healthops /opt/health/bin/health-report.sh
sudo chmod 750 /opt/health/bin/health-report.sh
sudo -u healthsvc /opt/health/bin/health-report.sh # اجرای دستی برای تست
ls -l /srv/health/
head -5 /srv/health/report.html

```

پنج نکته‌ی «سطح تولید» در این اسکرپت که در مصاحبه می‌پرسند:

نکته	چرا
<code>set -euo pipefail</code>	توقف روی خطا، خطای متغیر تعریف نشده، خطای وسط pipe
<code>trap cleanup EXIT</code>	فایل موقت حتی در خطا پاک شود (نبود آن = پر شدن <code>/srv/health</code> از فایل‌های نیمه‌کاره)
<code>flock -n</code>	اگر اجرای قبلی طول کشیده، این نوبت skip شود — نه اینکه دو نسخه همزمان بنویسند
نوشتن در temp و <code>mv -f</code>	جایگزینی اتمیک: خواننده هرگز فایل نیمه‌نوشته نمی‌بیند
<code>sort -u + ss -tulnH</code>	<code>-H</code> هدر را حذف می‌کند تا خروجی برای اسکریپت/پارس تمیز باشد

تست دستی این نکات: دو نسخه همزمان اجرا کن (`sudo -u healthsvc /opt/.../health-report.sh & sudo -u`)
`(healthsvc /opt/.../health-report.sh` — یکی باید پیام «already running» بدهد. سپس فایل را وسط اجرا نگاه کن؛
همیشه نسخه‌ی کامل قبلی را می‌بینی.

(۳) گام ۳ — سرویس و تایمر systemd (روز ۵)

۳.۱ — سرویس تولید گزارش (oneshot):

```
sudo tee /etc/systemd/system/health-report.service >/dev/null <<'UNIT'
[Unit]
Description=Server health report generator
Documentation=man:systemd.service(5)
After=network-online.target

[Service]
Type=oneshot
User=healthsvc
Group=healthops
WorkingDirectory=/opt/health
ExecStart=/opt/health/bin/health-report.sh
Nice=10
IOSchedulingClass=idle
UMask=0027
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=yes
ReadWritePaths=/srv/health
UNIT
```

۳.۲ — تایمر هر ۱۰ دقیقه:

```

sudo tee /etc/systemd/system/health-report.timer >/dev/null <<'UNIT'
[Unit]
Description=Run the health report every 10 minutes

[Timer]
OnBootSec=2min
OnUnitActiveSec=10min
AccuracySec=30s
Persistent=true
Unit=health-report.service

[Install]
WantedBy=timers.target
UNIT

```

۳.۳ – فعال‌سازی و وریفای:

```

sudo systemd-analyze verify /etc/systemd/system/health-report.service
sudo systemctl daemon-reload
sudo systemctl enable --now health-report.timer
systemctl status health-report.timer --no-pager
systemctl list-timers health-report.timer --no-pager
sudo systemctl start health-report.service # اجرای فوری برای تست
journalctl -u health-report -n 20 --no-pager
systemctl show health-report.service -p Type -p User -p Nice -p ReadWritePaths

```

توضیح کلیدواژه‌ها (این‌ها را باید بتوانی بی‌نگاه توضیح بدهی):

کلیدواژه	معنا در این لابراتوار
Type=oneshot	اسکرپت اجرا می‌شود، تمام می‌شود، سرویس «inactive (dead)» می‌شود – درست برای کار یک‌باره
OnBootSec=2min	۲ دقیقه بعد از بوت، یک اجرا (خارج از چرخه نیست ولی تضمین اجرای پس از بوت)
OnUnitActiveSec=10min	هر ۱۰ دقیقه بعد از آخرین اجرای سرویس، دوباره اجرا کن
Persistent=true	اگر سرور خاموش بود و اجرا از قلم افتاد، بعد از بوت جبران کن
AccuracySec=30s	تحمل ۳۰ ثانیه برای هم‌زمان‌نکردن بار روی سرورهای متعدد
IOSchedulingClass=idle / Nice	گزارش‌گیری هرگز سرویس اصلی را کند نکند
UMask=0027	فایل‌های ساخته‌شده‌ی سرویس به‌صورت پیش‌فرض گروه‌خواندنی و نه‌بیشتر
ReadWritePaths=/srv/ + ProtectSystem=strict health	کل فایل‌سیستم فقط‌خواندنی می‌شود به‌جز همان مسیری که باید بنویسد – دقیق‌ترین شکل سخت‌سازی
PrivateTmp=yes	/tmp خصوصی برای همین سرویس (سرویس‌ها همدیگر را نمی‌بینند)
NoNewPrivileges=yes	هیچ فرایند فرزند نمی‌تواند دسترسی بالاتر بگیرد (SUID بی‌اثر می‌شود)

```

sudo tee /etc/systemd/system/health-http.service >/dev/null <<'UNIT'
[Unit]
Description=Health dashboard (read-only HTTP on 8080)
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=healthsvc
Group=healthops
ExecStart=/usr/bin/python3 -m http.server 8080 --bind 0.0.0.0 --directory /srv/health
Restart=on-failure
RestartSec=3
NoNewPrivileges=yes
PrivateTmp=yes
ProtectSystem=strict
ProtectHome=yes
MemoryMax=128M
CPUQuota=25%

[Install]
WantedBy=multi-user.target
UNIT

sudo systemctl daemon-reload
sudo systemctl enable --now health-http.service
systemctl status health-http --no-pager | head -12
ss -tlnp | grep :8080
curl -s -o /dev/null -w 'code=%{http_code} time=%{time_total}s\n' http://127.0.0.1:8080/report.html

```

نکات مصاحبه‌ای این گام:

- `RestartSec=3` + `Restart=on-failure` یعنی اگر فرایند کرش کند، خودش برمی‌گردد؛ `kill` را تست کن (پایین‌تر).
- `CPUQuota` / `MemoryMax` = محدودسازی cgroup: یک سرویس جانبی هرگز CPU/RAM سرور اصلی را نمی‌خورد.
- `--directory /srv/health` یعنی وب‌سرور فقط همان پوشه را می‌بیند؛ لازم نیست در `/var/www` چیزی کپی شود.
- چون فقط گزارش استاتیک سرو می‌شود، `python -m http.server` کافی است؛ برای تولید واقعی: nginx با `auth_basic` یا پشت reverse proxy و TLS.

⚠ **هشدار:** `http.server` احراز هویت ندارد. در این لابراتوار با فایروال (گام ۵) محدودش می‌کنیم؛ در محیط واقعی هرگز سرویس بدون auth را باز نکن.

```
sudo tee /etc/nftables-health.conf >/dev/null <<'NFT'
#!/usr/sbin/nft -f
flush ruleset

table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;

    ct state established,related accept
    iif lo accept

    # ICMP لازم برای عیب‌یابی (فقط از شبکه‌ی مدیریتی)
    ip saddr 10.20.0.0/24 icmp type echo-request accept

    # SSH فقط از شبکه‌ی مدیریتی
    ip saddr 10.20.0.0/24 tcp dport 22 accept

    # داشبورد سلامت فقط از شبکه‌ی مدیریتی
    ip saddr 10.20.0.0/24 tcp dport 8080 accept

    counter log prefix "DROP-IN: " drop
  }

  chain forward { type filter hook forward priority 0; policy drop; }
  chain output { type filter hook output priority 0; policy accept; }
}
NFT

بدون اعمال (check) فقط بررسی (۱) # sudo nft -c -f /etc/nftables-health.conf
اعمال (۲) # sudo nft -f /etc/nftables-health.conf
sudo nft list ruleset | head -30
```

قبل از اعمال: مطمئن شو آدرس واقعی نشست SSH تو داخل `10.20.0.0/24` است (یا شبکه را عوض کن)، و یک نشست SSH دوم باز نگه دار. با `policy drop` یک اشتباه کوچک = قفل شدن بیرون. **پایدارسازی:** فایل بالا را در `/etc/nftables.conf` قرار بده (یا `include` کن) و `sudo systemctl enable --now nftables` تا در بوت اعمال شود.

تست از دو طرف (این جدول خودش یک سؤال مصاحبه است):

از کجا	فرمان	نتیجه‌ی انتظاری
شبکه‌ی مدیریتی	<code>nc -zv 10.20.0.10 8080</code>	<code>succeeded</code>
شبکه‌ی مدیریتی	<code>curl -s http://10.20.0.10:8080/report.html</code> (سه خط اول)	HTML گزارش
شبکه‌ی نامجاز	<code>nc -zv 10.20.0.10 8080</code>	timeout / refused
خود سرور	<code>sudo nft list ruleset -a</code> (ردیف‌های counter)	شمارنده‌ی drop بالا رفتن
شبکه‌ی مدیریتی	<code>nmap -Pn -p 22,80,8080 10.20.0.10</code>	۲۲ و ۸۰۸۰ باز، بقیه فیلتر

۶ گام ۶ – SSH کلیدمحور و قفل کردن دسترسی (روز ۶)

```
# روی کلاینت (ادمین)
ssh-keygen -t ed25519 -a 100 -C "ops-admin-2026" -f ~/.ssh/id_srv1
ssh-copy-id -i ~/.ssh/id_srv1.pub alice@10.20.0.10
ssh -i ~/.ssh/id_srv1 alice@10.20.0.10 'hostname; id; systemctl is-active health-report.timer'
```

```
# روی سرور: سخت‌سازی
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.$(date +%F)
sudo sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
grep -E '^(PasswordAuthentication|PermitRootLogin|PubkeyAuthentication)' /etc/ssh/sshd_config
sudo sshd -t && sudo systemctl reload ssh # (در RHEL: sshd)
```

```
# تست در نشست دوم (نه همان نشستی که توش هستی!)
ssh -o PreferredAuthentications=publickey alice@10.20.0.10 'echo key-ok'
ssh -o PreferredAuthentications=password alice@10.20.0.10 'echo should-fail' # باید رد شود
```

۷ گام ۷ – پذیرش نهایی (Acceptance Test)

این ۱۲ چک را به‌ترتیب اجرا کن و خروجی هرکدام را ثبت کن (این همان چیزی است که در تیکت/مصاحبه نشان می‌دهی):

```

# کاربر و گروه ۱
id healthsvc ; getent group healthops ; sudo -u healthsvc -s /bin/bash -c 'echo shell-works'
# اسکریپت ۲
sudo -u healthsvc /opt/health/bin/health-report.sh
# تایمر ۳
systemctl is-enabled health-report.timer ; systemctl is-active health-report.timer
systemctl list-timers health-report.timer --no-pager | head -3
# فایل گزارش: مالکیت/مجوز ۴
ls -l /srv/health/report.html ; getfacl -p /srv/health/report.html | head -8
# ops (با کاربر ops) خواندن توسط تیم ۵
sudo -u alice test -r /srv/health/report.html && echo "ops can read"
sudo -u nobody test -r /srv/health/report.html 2>/dev/null && echo "LEAK!" || echo "others cannot read"
# داشبورد ۶
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8080/report.html
# ریاستارت خودکار سرویس وب ۷
sudo systemctl kill -s SIGKILL health-http ; sleep 5 ; systemctl is-active health-http
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8080/report.html
# محدودیت منابع ۸
systemctl show health-http -p MemoryMax -p CPUQuota -p Restart
# روی سرویس گزارش systemd سختسازی ۹
systemctl show health-report -p ProtectSystem -p ReadWritePaths -p NoNewPrivileges -p UMask
# فایروال ۱۰
sudo nft list ruleset | sed -n '1,20p'
# لاگها ۱۱
journalctl -u health-report --since "-30 min" --no-pager | tail -5
journalctl -u health-http -b --no-pager | tail -3
# آزمون بوت: ریبت و بعد از بالا آمدن ۱۲
sudo reboot
# پس از بوت:
systemctl is-active health-report.timer health-http ; ls -l /srv/health/report.html

```

جاهای خالی این جدول را بعد از اجرا پر کن (خروجی واقعی خودت):

#	چک	انتظار	خروجی من
۱	<code>id healthsvc</code>	uid سیستمی + shell nologin	
۲	<code>list-timers</code>	≤ ۱۰ NEXT دقیقه	
۳	<code>ls -l report.html</code>	<code>-rw-r----- healthsvc healthops</code>	
۴	<code>getfacl</code>	<code>group:ops:r-x</code>	
۵	<code>curl</code> از شبکه‌ی مدیریتی	200	
۶	<code>nc</code> از شبکه‌ی نامجاز	timeout	
۷	<code>systemctl kill SIGKILL health-http</code>	سرویس خودکار بالا آمد	
۸	reboot	هر دو سرویس فعال + گزارش تازه	

۸) تمرین‌های خراب‌کن و درست‌کن (Break & Fix)

#	خرابی که خودت می‌سازی	علامت	تشخیص و رفع
۱	<code>sudo chown root:root /srv/health</code> (مالکیت را برگردان)	سرویس گزارش با خطای نوشتن می‌افتد	→ <code>journalctl -u health-report -n 20</code> → <code>Permission denied</code> مالکیت را به <code>healthsvc:healthops</code> برگردان
۲	<code>ReadWritePaths=</code> را از unit حذف کن (یا مسیر را عوض کن)	خطای <code>Read-only file system</code> با وجود <code>ProtectSystem=strict</code>	<code>systemctl show -p</code> و <code>systemctl cat</code> <code>ReadWritePaths</code> ؛ مسیر درست را برگردان
۳	<code>User=</code> را به کاربر ناموجود تغییر بده	<code>Failed to determine user credentials</code> و کد ۲۱۷	<code>journalctl -u health-report</code> ؛ کاربر یا unit را اصلاح کن
۴	در <code>ExecStart</code> مسیر را یک حرف غلط بنویس	<code>status=203/EXEC</code>	اجرای دستی همان فرمان با همان کاربر؛ اصلاح مسیر مطلق
۵	پورت ۸۰۸۰ را با <code>python3 -m http.server 8080</code> اشغال کن، بعد سرویس را ری‌استارت کن	<code>health-http</code> در چرخه‌ی <code>failed</code> ، خطای <code>Address already in use</code>	<code>ss -tlnp</code> (فیلتر روی ۸۰۸۰)؛ کشتن دارنده؛ ری‌استارت سرویس
۶	<code>systemctl disable health-report.timer</code> و ری‌بوت	بعد از بوت، گزارش قدیمی می‌ماند	<code>enable --</code> → <code>systemctl is-enabled now</code>
۷	قاعده‌ی ۸۰۸۰ را از فایروال حذف کن	از شبکه‌ی مدیریتی هم <code>timeout</code>	<code>nft list ruleset</code> + شمارنده‌ی <code>drop</code> ؛ برگرداندن قاعده
۸	روی <code>report.html</code> <code>chmod 600</code> (به جای ۶۴۰)	تیم <code>ops</code> نمی‌تواند بخواند ولی سرویس سالم است	<code>getfacl</code> ؛ اصلاح به <code>640</code> + <code>ACL</code> پیش‌فرض پوشه

هر تمرین را قبل از دیدن راه‌حل، خودت تشخیص بده: علامت → ابزار → ریشه → رفع → پیشگیری. این چرخه‌ی پنج‌مرحله‌ای، همان چیزی است که در مصاحبه از تو می‌خواهند.

۹) آزمون‌های منتخب هفته ۲ (با پاسخنامه)

۹.۱ آزمون کوتاه – ۱۵ سؤال

- تفاوت نقش `flock` و `trap` در اسکریپت این لابراتوار چیست؟
- چرا گزارش در فایل موقت نوشته می‌شود و بعد `mv` می‌شود؟
- `UMask=0027` در unit چه اثری روی فایل‌های ساخته‌شده دارد؟
- تفاوت `Type=oneshot` و `Type=simple` – و کدام برای این لابراتوار و چرا؟
- تفاوت `OnUnitActiveSec=10min` و `OnCalendar=*:0/10` ؟
- `Persistent=true` برای چه سناریویی است؟
- اگر `ReadWritePaths` را فراموش کنیم، با `ProtectSystem=strict` چه خطایی می‌گیریم؟
- `NoNewPrivileges=yes` جلوی چه حمله/سوءاستفاده‌ای را می‌گیرد؟
- چرا کاربر سرویس را با `-s /usr/sbin/nologin` و `-r` ساختیم؟
- تفاوت `flock -n` و `flock` بدون `-n` ؟
- کدهای `203/EXEC` و `200/CHDIR` در `systemd` چه معنایی دارند؟

12. چرا در `ss -tuln` سوئیچ `-H` می‌گذاریم؟
13. `getfacl` نشان می‌دهد `mask::r-x` – این یعنی چه و چه وقت مشکل‌ساز می‌شود؟
14. تفاوت `systemctl reload` و `restart` و اینکه در این لابراتوار برای تغییر فایل گزارش کدام بی‌ربط است؟
15. اگر بخواهی مطمئن شوی «هیچ ترافیک ناخواسته‌ای وارد نشده»، از ابزارهای هفته ۲ چه چیزی را چطور می‌گذاری؟

۹.۲ آزمون چندگزینه‌ای – ۱۵ سؤال

- ۱) نام کاربر سیستمی بدون شل ورود برای سرویس: الف) `useradd -m -s /bin/bash svc` ب) `useradd -r -s /usr/sbin/` ج) `nologin svc` د) `adduser svc` (ب) `usermod -L svc`
- ۲) کدام گزینه سرویس را در برابر کرش مقاوم می‌کند؟ الف) `Type=oneshot` ب) `Restart=on-failure` + `RestartSec` ج) `WantedBy=multi-user.target` د) `After=network.target`
- ۳) تایمری که هم در بوت و هم هر ۱۰ دقیقه اجرا شود: الف) فقط `OnCalendar` ب) `OnBootSec` + `OnUnitActiveSec` ج) فقط `Persistent=true` د) `AccuracySec=30s`
- ۴) `ProtectSystem=strict` می‌کند: الف) فایل سیستم را فقط خواندنی، به‌جز `ReadWritePaths` ب) کاربر را محدود می‌کند ج) لاگ را می‌بندد د) پورت‌ها را می‌بندد
- ۵) برای دیدن زمان اجرای بعدی تایمر: الف) `systemctl status` ب) `systemctl list-timers` ج) `journalctl -u timer` د) `systemd-analyze blame`
- ۶) جای `journalctl -u health-report -f`: الف) تعقیب زنده‌ی لاگ سرویس ب) پاک‌کردن لاگ ج) فقط خطاهای سرویس د) خلاصه‌ی مصرف
- ۷) برای «فقط گروه ops بخواند» کدام بهتر است؟ الف) `chmod 755` ب) `chmod 644` ج) `ACL`: `setfacl -m g:ops:r-x` د) `chown ops`
- ۸) جای `ss -tlnp`: الف) سوکت‌های UDP ب) سوکت‌های TCP گوش‌دهنده با فرایند ج) اتصال‌های بسته د) جدول ARP
- ۹) برای اعمال یک سیاست فایروال با خطر قفل‌شدن، اول چه می‌زنی؟ الف) `nft -f` مستقیم ب) `nft -c -f` (بررسی نحوی) و باز نگه‌داشتن نشست دوم ج) `iptables -F` د) ریبوت
- ۱۰) کدام گزینه در «deny» nftables ضمیمه را می‌سازد؟ الف) `policy accept` ب) `policy drop` ج) `counter` د) `log`
- ۱۱) برای دیدن اینکه چند بسته توسط قاعده‌ی drop خورده، از چه استفاده می‌کنی؟ الف) `counter` در قاعده + `nft list ruleset -a` ب) `ss -s` ج) `ss -s` د) `journalctl -k`
- ۱۲) تفاوت `ssh -o PreferredAuthentications=publickey` در تست چیست؟ الف) کلید را نادیده می‌گیرد ب) فقط احراز هویت با کلید را امتحان می‌کند ج) رمز را اجبار می‌کند د) پورت را عوض می‌کند
- ۱۳) `systemd-analyze verify <unit>` چه می‌کند؟ الف) سرویس را اجرا می‌کند ب) unit را از نظر خطای نحوی/منطقی بررسی می‌کند ج) زمان بوت می‌دهد د) لاگ را می‌خواند
- ۱۴) بهترین مکان برای لاگ خروجی اسکریپت یک سرویس بدون تنظیم اضافه: الف) فایل در `/tmp` ب) `journald` (خروجی استاندارد سرویس) ج) `dmesg` د) `wtmp`
- ۱۵) برای «مقاوم‌کردن» گزارش‌گیری در برابر هم‌زمانی (concurrency) در چند سرور، کدام مفید است؟ الف) `RandomizedDelaySec` ب) `Nice=10` ج) `Restart=always` د) `AccuracySec=0`

۱۰) چالش‌های عملی تکمیلی (۵ مورد)

#	چالش	معیار قبولی
۱	گزارش را به JSON هم تولید کن (report.json) و در داشبورد لینکش کن	فایل با jq بدون خطا پارس شود
۲	اگر دیسکی بالای ۹۰٪ بود، اسکریپت کد خروجی ۱ بدهد و در journald هشدار ثبت شود	ردیف‌های WARN در journalctl -u health-report
۳	یک آلارم سبک بساز: اگر سرویس health-http down بود، با wall / mail هشدار بده (در تایمر دوم)	با systemctl stop health-http هشدار فعال شود
۴	دسترسی داشبورد را از «شبکه‌ی مدیریتی» به «شبکه‌ی مدیریتی + TLS» ارتقا بده (nginx + self-signed)	curl -k https://10.20.0.10/ جواب دهد
۵	با rsync یک نسخه از گزارش‌ها را روزانه به سرور دیگر بفرست و با sha256sum یکپارچگی را چک کن	هش دو طرف یکسان

۱۱) پاسخنامه

۱۱.۱ پاسخ آزمون کوتاه (۱۵ سؤال)

1. flock از اجرای همزمان دو نسخه جلوگیری می‌کند (قفل فایل)؛ trap پاک‌سازی را در هر نوع خروج (خطا/سیگنال/پایان) تضمین می‌کند.
2. جایگزینی اتمیک: خواننده (وب‌سرور/تیم ops) هرگز فایل نیمه‌نوشته یا خالی نمی‌بیند؛ اگر اسکریپت وسط کار بمیرد، نسخه‌ی قبلی سالم می‌ماند.
3. فایل‌های ساخته‌شده‌ی سرویس با ماسک 0027 ساخته می‌شوند: گروه r-x ... نه، دقیقاً 640 برای فایل و 750 برای پوشه →
other:--- , group:r-- , owner:rw-
4. oneshot برای کاری است که انجام می‌شود و تمام (یک اجرا، پایان)؛ simple برای فرایندی که باید زنده بماند. در این لابراتوار: گزارش = oneshot ، داشبورد = simple
5. OnUnitActiveSec نسبی است (۱۰ دقیقه بعد از آخرین اجرای سرویس)؛ OnCalendar مطلق و ساعت‌محور است (00:00, ..., 00:10). اگر اجرا طول بکشد، دومی می‌تواند روی اجرای قبلی بیفتد؛ اولی نه.
6. برای سروری که خاموش/ریبوت می‌شود: جبران اجرای از قلم‌افتاده بعد از بوت (مثلاً گزارش ساعت ۳ که سرور در نگهداری بوده).
7. خطای نوشتن مثل Failed to write file ...: Read-only file system (و سرویس failed می‌شود).
8. جلوی ارتقای دسترسی از طریق بیت‌های SUID/قابلیت‌های فایل در فرایند سرویس (و فرزندانش) را می‌گیرد؛ یعنی حتی اگر مهاجم به کد سرویس برسد، راه SUID بسته است.
9. -r کاربر سیستمی است (UID پایین، بدون انقضای رمز، مناسب سرویس)؛ nologin اجازه‌ی ورود تعاملی نمی‌دهد، پس حتی با رمز لو رفته، مهاجم شل نمی‌گیرد.
10. -n غیرمسدودکننده است: اگر قفل گرفته شده باشد فوراً شکست می‌خورد (در کرون یعنی skip)؛ بدون -n منتظر می‌ماند تا قفل آزاد شود (می‌تواند صف بسازد و اجرای انبوه ایجاد کند).
11. 203/EXEC = نتوانست فایل اجرایی را اجرا کند (مسیر/مجوز/شبنگ)؛ 200/CHDIR = نتوانست به WorkingDirectory برود (وجود ندارد یا دسترسی نیست).
12. حذف خط هدر در خروجی ss تا پارس/مرتب‌سازی اسکریپتی تمیز باشد.

13. **mask** سقف مؤثر مجوزهای ورودی‌های ACL است؛ اگر **r** لازم داری ولی **mask** فقط **r-x** ... دقیق‌تر: **mask** سقف مجوزهای گروه‌های نام‌دار/کاربران ACL است؛ اگر **mask r-x** باشد و به کاربر **rw** داده باشی، مؤثر همان **r-x** می‌شود – شایع‌ترین دلیل «ACL اعمال نشد».

14. **reload** کانفیگ را با سیگنال (HUP) بازخوانی می‌کند؛ **restart** فرایند را می‌کشد و بالا می‌آورد. فایل گزارش داده‌ی زمان‌اجراست، نه کانفیگ؛ برای آن هیچ‌کدام لازم نیست – تایمر خودش هر ۱۰ دقیقه تازه‌سازی می‌کند (**systemctl start health-report.service** برای اجرای فوری).

15. با **nft** شمارنده روی قاعده‌ها (**counter log prefix**) + بازبینی دوره‌ای **journalctl -k** , **journalctl -u ssh** برای تلاش‌های ورود، و **tcpdump -nn -i any -c 200** روی پورت‌های حساس؛ در سطح شبکه هم ACL/ایزوله‌سازی سگمنت‌ها.

۱۱.۲ کلید چندگزینه‌ای

س	پاسخ	س	پاسخ	س	پاسخ
۱	ب	۶	الف	۱۱	الف
۲	ب	۷	ج	۱۲	ب
۳	ب	۸	ب	۱۳	ب
۴	الف	۹	ب	۱۴	ب
۵	ب	۱۰	ب	۱۵	الف

۱۱.۳ ده سؤال مصاحبه (مرتبط با همین لابراتوار، با پاسخ مدل)

۱) یک سرویس را چطور «امن» و «مقاوم» می‌سازی؟ (کامل جواب بده) کاربر سیستمی اختصاصی و بدون شل (**-r -s nologin**)، مالکیت و مجوز دقیق روی پوشه‌های داده/لاگ، **unit** با **Group / User**، سخت‌سازی **systemd** (**PrivateTmp** , **NoNewPrivileges** , **ProtectSystem=strict** + **ReadWritePaths** , **ProtectHome** , **UMask=0027**)، محدودیت منابع (**MemoryMax** , **CPUQuota** , **LimitNOFILE**)، **Restart=on-failure** + **RestartSec**، لاگ در **journald**، فایروال میزبان برای پورت سرویس، و پایش/هشدار روی وضعیت سرویس و مصرف منابع. در نهایت: تست‌های پذیرش و مستندسازی.

۲) چرا از **systemd timer** به جای **cron** استفاده کردی؟ لاگ یکپارچه در **journal**، کنترل منابع و اولویت (**IOSchedulingClass**)، **Nice**، وابستگی و ترتیب (**After**)، **Persistent=true** برای جبران اجرای ازقلم‌افتاده، **RandomizedDelaySec** برای پخش بار، و مشاهده‌ی ساده‌ی زمان اجرای بعدی با **list-timers**. کرون را برای سازگاری با اسکریپت‌های قدیمی و سیستم‌های بدون **systemd** نگه می‌دارم.

۳) کاربر می‌گوید گزارش قدیمی است و به‌روز نمی‌شود. چه می‌کنی؟ **systemctl list-timers health-report.timer** (اجرای بعدی/قبلی)، **systemctl status health-report.timer** (؟is-enabled)، **systemctl status health-report.service** (؟failed)، **sudo -u healthsvc /opt/** سپس اجرای دستی با همان کاربر **journalctl -u health-report --since "-1h"**، **health/bin/health-report.sh**؛ اگر دستی موفق → مجوز/مسیر/قفل. در پایان، علت را مستند و پایش را روی «سن فایل گزارش» اضافه می‌کنم.

۴) تفاوت **ProtectSystem=strict** و **ReadWritePaths / ReadOnlyPaths** را توضیح بده. **ProtectSystem=strict** کل سلسله‌مراتب فایل‌سیستم را برای سرویس فقط‌خواندن می‌کند (به‌جز **/sys** , **/proc** , **/dev**) که به‌شکل محدود مدیریت می‌شوند؛ سپس با **ReadWritePaths=** نقاط نوشتن را استثنا می‌کنی (اینجا **/srv/health**). **ReadOnlyPaths** برعکس، مسیرهای مشخصی را (در حالت‌های غیر **strict**) فقط‌خواندن می‌کند. نتیجه: کمترین دسترسی لازم – اصل **least privilege** در سطح فایل‌سیستم.

۵) اگر مهاجم به کد سرویس وب نفوذ کند، چه چیزهایی جلوی گسترش را می‌گیرد؟ (دفاع لایه‌ای) کاربر بدون امتیاز **healthsvc**، **NoNewPrivileges=yes** (SUID بی‌اثر)، **ProtectSystem=strict** (نمی‌تواند باینری/کانفیگ سیستم را تغییر دهد)، **ProtectHome**، **PrivateTmp**، **MemoryMax** (جلوگیری از مصرف انبوه)، فایروال میزبان با **policy drop** (خروج/ورود نامجاز بسته)، لاگ **journal** برای کشف، و در لایه‌ی شبکه: ACL/ایزوله‌سازی سگمنت و پایش. نکته‌ی مهم: **http.server** احراز هویت ندارد، پس یا پشت **reverse proxy** با **auth** می‌رود یا فقط از شبکه‌ی مدیریتی قابل دسترسی است.

۶) **ACL** را کجا ترجیح می‌دهی به **chmod** و چرا؟ وقتی چند گروه/کاربر با نیازهای متفاوت روی یک پوشه‌اند و مدل سه‌گانه‌ی POSIX کافی نیست (مثلاً: مالک سرویس، گروه ops خواندن، کاربر audit فقط خواندن یک فایل). **chmod** به «همه» اجازه می‌دهد یا گروه‌های اضافی می‌سازد؛ **ACL** دقیق و قابل بازرسی است (**getfacl**) و با **ACL** پیش‌فرض (**-d**) روی پوشه، فایل‌های جدید خودکار همان مجوز را می‌گیرند.

۷) تفاوت «اجرای اتمیک گزارش» و «نوشتن مستقیم» را با یک سناریو بگو. اگر مستقیم روی **report.html** بنویسی و کاربر/وب‌سرور همان لحظه بخواند، ممکن است فایل نیمه‌کاره یا خالی ببیند (و داشبورد خراب شود). با نوشتن در فایل موقت و **mv -f** (که در همان فایل‌سیستم اتمی است)، خواننده همیشه یا نسخه‌ی قبلی کامل را می‌بیند یا نسخه‌ی جدید کامل – هرگز بین آن‌ها.

۸) چطور مطمئن می‌شوی سرویس در بوت بالا می‌آید و «واقعاً» کار می‌کند؟ **systemctl is-enabled** (پیوند فعال)، **systemctl is-active** پس از ری بوت، **list-timers** برای اجرای زمان‌بندی‌شده، بررسی تازگی خروجی (**stat /srv/health/report.html**)، لاگ بوت (**journalctl -b -u health-report**)، و تست کاربردی (**curl** روی پورت). در سطح پیشرفته‌تر: یک آلارم که اگر فایل گزارش از ۳۰ دقیقه قدیمی‌تر شد هشدار بدهد – چون «سرویس active ولی خروجی بی‌فایده» یک شکست خاموش است.

۹) چرا **ss -tuln** و **nmap** هر دو را داری؟ چه تفاوتی دارند؟ **ss** دید درون میزبان است: چه چیزی روی چه آدرسی گوش می‌دهد (و با **-p** کدام فرایند). **nmap** دید از بیرون/شبکه است: چه چیزی عملاً از شبکه قابل دسترسی است (با فایروال، **ACL** و مسیر). تفاوت نتیجه‌ی این دو دقیقاً همان چیزی است که فایروال مسدود می‌کند – کلید تشخیص «سرویس باز است ولی از بیرون بسته».

۱۰) اگر بخواهی این لابراتوار را با **Ansible** خودکار کنی، چه می‌کنی؟ یک **playbook** با نقش‌های: **user/group** (ماژول **user**، **group**)، **copy / file** برای اسکریپت و **unit**ها (با **template** و handler برای **restart / daemon-reload**)، **acl** برای **ACL**، **systemd** برای **enabled: yes, state: started**، **ufw / nftables** برای فایروال، و **template / lineinfile** برای **sshd_config** + handler **reload ssh**. مزیت‌ها: **idempotent**، قابل اجرا روی چند سرور، و مستندسازی خودکار تغییرات؛ و برای تست، **--check** و **--diff**.

۱۲) چک‌لیست پایان این آزمایشگاه

- [] کاربر **healthsvc** با شل **nologin** ساخته شد و هیچ دسترسی اضافه ندارد
- [] اسکریپت گزارش اجرا می‌شود، **flock** / **trap** / جایگزینی اتمیک را تست کرده‌ام
- [] سرویس **oneshot** + تایمر ۱۰ دقیقه فعال و در بوت **enabled** است
- [] داشبورد روی ۸۰۸۰ سرو می‌دهد و محدودیت منابع دارد
- [] سخت‌سازی **systemd** (**ProtectSystem=strict**، **NoNewPrivileges**، **UMask**) را تست کرده‌ام
- [] فایروال با **policy drop** اعمال شده و از دو شبکه تستش کرده‌ام
- [] **SSH** کلیدمحور شد و ورود با رمز رد می‌شود
- [] ۱۲ چک پذیرش را با خروجی واقعی ثبت کرده‌ام
- [] هر ۸ تمرین **Break & Fix** را (حداقل ۵ تای آن) انجام داده‌ام
- [] آزمون‌ها را بدون پاسخنامه زده‌ام و بعد تطبیق داده‌ام
- [] به ۱۰ سؤال مصاحبه بلند جواب داده‌ام
- [] همه‌ی فایل‌ها (اسکریپت، **unit**ها، کانفیگ فایروال) را در یک مخزن **Git** کامیت کرده‌ام